



# Cookbook Memory

AI Agent Memory Harness — persistent memory for long-running coding agents

Codename **Cookbook Memory** · Team of 4 (Keith · Ken · Brent · Scott B.) · Two-week sprint · Five public benchmarks

Repository: [github.com/kenhuangus/agent-memory-harness](https://github.com/kenhuangus/agent-memory-harness) · Live: [kenhuangus.github.io/agent-memory-harness](https://kenhuangus.github.io/agent-memory-harness)

## 1 The problem

There is **no standard, model-agnostic layer** that decides **what to remember, where to store it, how to retrieve it fast**, and **how to keep it consistent over time** for long-running agents.

| DECISION   | WHAT IT MEANS       |
|------------|---------------------|
| What       | signal vs. noise    |
| Where      | the right store     |
| Fast       | no context flood    |
| Consistent | dedup, no conflicts |

## 2 Hypothesis & success criteria

**Hypothesis.** With the memory harness, **Haiku can close the gap to Opus 4.8** (which runs without memory) on established memory benchmarks.

**Success criterion.** On **at least two of the five** benchmarks, *Haiku + harness* beats the *Opus 4.8 no-memory* baseline across the four metrics, **without** blowing the efficiency budget (memory adds **< ~10%** context-token overhead).

| METRIC     | DEFINITION                                        |
|------------|---------------------------------------------------|
| Recency    | Is the freshest relevant memory ranked first?     |
| Efficiency | Tokens per retrieval; target under ~10% overhead. |
| Relevancy  | Retrieved items actually relate to the query.     |
| Accuracy   | Task correctness, memory on vs. off.              |

## 3 Technical approach

A **pluggable memory harness**: four modules over three indexed storage backends, wired into the open-source **OpenCode** agent and measured on public benchmarks.

## Four modules

- **Persistence layer (write).** Decides what/when/where to save; tags each item (timestamp, relevancy, session); versioning + dedup hints.
- **Intelligent router (dispatch).** Classifies each query and hits the *single best* backend. Rule-based, then learned.
- **Retrieval orchestrator (read).** Unified API: route, rank by `recency × relevancy`, de-dup, return a tight context.
- **Dreaming component (async).** Runs **while other agents sleep**: de-dup, resolve contradictions, session governance (must-know / must-do / blacklist), retention + pruning.

## Three storage backends (each indexed)

| BACKEND          | BEST FOR                  | INDEX                    |
|------------------|---------------------------|--------------------------|
| Markdown + YAML  | Literal recall            | Inverted keyword map     |
| SQLite + vectors | Semantic search           | Dense ANN (HNSW / FAISS) |
| Graph store      | Relationships & conflicts | Typed traversal          |

## Evaluation

Public benchmarks only — no home-grown evals. Per benchmark, a controlled grid runs Haiku + harness vs. Haiku, Opus 4.8 and Sonnet — prompts and task order fixed, every trajectory logged.

# 4 Scope

---

## In scope

- The memory harness: all four modules, end to end.
- Three indexed storage adapters behind one interface.
- The intelligent router (rule-based; learned-classifier hook).
- The async dreaming component: dedup, conflict resolution, governance, retention.
- OpenCode integration for memory write/read on each step.
- Eval harness over five benchmarks; baselines for Haiku, Opus 4.8, Sonnet.
- Metric definitions, a reproducible protocol, results dashboard.

## Out of scope (non-goals)

- Building our own benchmark or dataset — we use public ones.
- Training or fine-tuning any model.
- Production hardening: multi-tenant infra, auth, SLAs, polished UI.
- Distributed / scaled deployment — single-node is enough for the sprint.
- Validating non-coding agents — design generalizes, but not tested now.

## 5 Team & ownership — who owns what

Four parallel workstreams, locked behind a shared storage interface **frozen on Day 3**.

**The team:** **P1 Keith** (harness, OpenCode) · **P2 Ken** (benchmark) · **P3 Brent** (store, retrieve) · **P4 Scott B.** (dreaming).

| AREA / DELIVERABLE                                       | OWNER                | SUPPORTING |
|----------------------------------------------------------|----------------------|------------|
| Storage interface & memory-item schema                   | <b>P1 · Keith</b>    | P3         |
| Persistence layer (write path, tagging, versioning)      | <b>P1 · Keith</b>    | —          |
| Retrieval orchestrator (rank · dedup · return)           | <b>P1 · Keith</b>    | P3         |
| OpenCode integration (write/read on each step)           | <b>P1 · Keith</b>    | —          |
| Three storage backends + per-backend indexes             | <b>P3 · Brent</b>    | P1         |
| Intelligent router (rules → learned)                     | <b>P3 · Brent</b>    | P1         |
| Backend performance testing                              | <b>P3 · Brent</b>    | P2         |
| Dreaming worker (dedup, conflict, governance, retention) | <b>P4 · Scott B.</b> | P1, P3     |
| Memory semantics ("what is good memory")                 | <b>P4 · Scott B.</b> | P2         |
| Datasets, loaders & trajectory logging                   | <b>P2 · Ken</b>      | P4         |
| Metric defs, shared run harness & cost gates             | <b>P2 · Ken</b>      | P4         |
| Results + cost dashboard, stats, aggregation             | <b>P2 · Ken</b>      | —          |
| Run SWE-Bench-CL eval (own key)                          | <b>P1 · Keith</b>    | —          |
| Run LongMemEval eval (own key)                           | <b>P2 · Ken</b>      | —          |
| Run SWE-ContextBench eval (own key)                      | <b>P3 · Brent</b>    | —          |
| Run MemoryAgentBench eval (own key)                      | <b>P4 · Scott B.</b> | —          |
| Run ContextBench eval (own key)                          | <b>P3 · Brent</b>    | —          |
| Final end-to-end integration                             | <b>All</b>           | —          |

- **P1 · Keith — Harness Architecture & OpenCode Integration.** Critical path — the contracts everyone builds against.
- **P2 · Ken — Evaluation Infrastructure & Coordination.** Shared runner, metrics, dashboard — and captains one benchmark.
- **P3 · Brent — Storage Implementation & Router.** Three fast backends + the dispatch layer; captains the two retrieval benchmarks.
- **P4 · Scott B. — Dreaming Component & Memory Governance.** The offline engine that keeps memory clean.

**Dividing the eval runs (API cost & time)**

~20 runs (5 benchmarks × 4 configs) is too much cost and time for one person on one key. Each teammate **captains** the benchmark(s) that stress their own component, on their **own API budget**. Any collaborator can also launch a run from the **GitHub Actions** tab (free offline by default).

| BENCHMARK        | CAPTAIN       | WHY THEM                                                          |
|------------------|---------------|-------------------------------------------------------------------|
| SWE-Bench-CL     | Keith (P1)    | Drives the coding agent end-to-end — his OpenCode wheelhouse.     |
| LongMemEval      | Ken (P2)      | Eval lead; recency / temporal reasoning.                          |
| SWE-ContextBench | Brent (P3)    | Cross-task context reuse exercises his retrieval + router.        |
| MemoryAgentBench | Scott B. (P4) | Tests conflict resolution — his dreaming component.               |
| ContextBench     | Brent (P3)    | Retrieval-quality (gold contexts) — the retrieval/router he owns. |

Ken owns the shared runner — `run(benchmark, model, memory) → metrics` — and aggregates all results. Controls: sharded keys (one per captain); cheapest-first + early-exit (Opus last, confirm-only); dev-slice → full; cache + resume; hard per-benchmark budget gates; no-memory baselines in week 1.

## 6 Two-week timeline

---

### Person 1 · Keith — Harness Architecture & OpenCode Integration

- **Week 1:** Three-module abstraction & storage interface. Persistence layer (write, versioning, tagging). Instrument OpenCode to write memory each step.
- **Week 2:** Retrieval orchestrator (rank → dedup → return); wire in Brent's router; end-to-end task. **Captain the SWE-Bench-CL runs.**

### Person 2 · Ken — Evaluation Infrastructure & Coordination

- **Week 1:** Parse all five datasets; loaders + trajectory logging (with Scott B.). Define the metrics; build the shared run harness + cost tracker.
- **Week 2: Captain the LongMemEval runs.** Aggregate every captain's results; stats + results & cost dashboard; document the protocol.

### Person 3 · Brent — Storage Implementation & Router

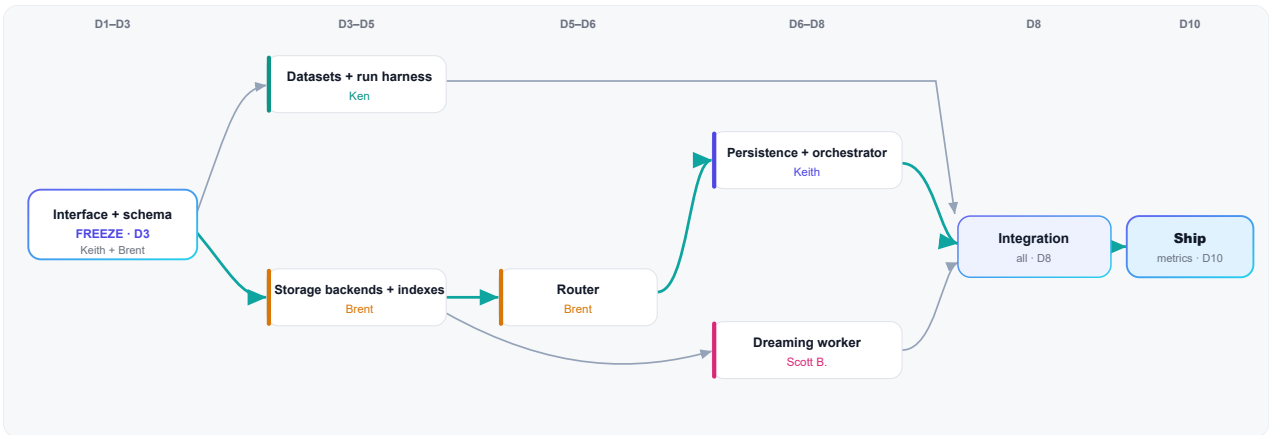
- **Week 1:** SQLite + vector pipeline (HNSW/FAISS). Markdown store + inverted index. Graph store (Neo4j) + traversal index.
- **Week 2:** Adapters behind Keith's interface; implement the router; performance-test each backend. **Captain the SWE-ContextBench & ContextBench runs.**

### Person 4 · Scott B. — Dreaming Component & Memory Governance

- **Week 1:** Dedup (exact / semantic / near-dup). Conflict detection + reconciliation. Session-filter & blacklist. Co-build trajectory logging with Ken.
- **Week 2:** Async scheduler that runs while agents sleep; must-know / must-do extraction; retention & pruning; observability; integration with Keith & Brent. **Captain the MemoryAgentBench runs.**

## 7 Dependency map & critical path

The Day-3 interface freeze unblocks every stream; the bold chain is the critical path to ship.



Teal = critical path (freeze → backends → router → orchestrator → integration → ship). Grey = supporting dependencies.

- **Keith + Brent (D1–D3):** co-author and freeze the storage interface + schema, so persistence and adapters share one contract.
- **Brent → Keith:** the router lands before the orchestrator can route in week 2.
- **Ken → all (by D5):** datasets, logging and the shared run harness ready, so baselines can start ~D4–D6.
- **Keith + Brent → Scott B.:** the dreaming worker integrates once real storage exists (D8+).
- **Sharded keys:** each captain on a separate API budget — baselines week 1, treatment week 2.

**Top risk is semantic, not structural:** defining *what counts as "good memory."* Ken and Scott B. must align on memory semantics in week 1, or the harness stores everything and retrieves nothing useful.

## 8 Milestones

| DAY | MILESTONE                                                                                                                          |
|-----|------------------------------------------------------------------------------------------------------------------------------------|
| D3  | <b>Contract freeze</b> — storage interface + memory-item schema locked; metric definitions agreed.                                 |
| D5  | <b>End of week 1</b> — three backends read/write; shared run harness + loaders ready; no-memory baselines started on sharded keys. |
| D8  | <b>Integration start</b> — router + orchestrator connected; dreaming worker running against real stores.                           |
| D10 | <b>Ship</b> — all benchmark shards complete (Haiku + harness); four metrics aggregated vs. baselines.                              |

## 9 Benchmarks & metrics

| BENCHMARK               | PRIMARY METRICS                                   | REFERENCE        |
|-------------------------|---------------------------------------------------|------------------|
| <b>MemoryAgentBench</b> | Relevancy, Accuracy, conflict resolution          | arXiv 2507.05257 |
| <b>LongMemEval</b>      | Recency, Accuracy, temporal reasoning             | arXiv 2410.10813 |
| <b>SWE-ContextBench</b> | Efficiency, Relevancy, Accuracy                   | arXiv 2602.08316 |
| <b>SWE-Bench-CL</b>     | Relevancy, Accuracy, continual learning           | arXiv 2507.00014 |
| <b>ContextBench</b>     | Relevancy, Efficiency, retrieval recall/precision | arXiv 2602.05892 |

Full descriptions, dataset links and the metric-mapping matrix live on the project site's **Benchmarks** page.

## 10 Collaboration & deliverables

Four developers, one repo, no conflicts: **prd.md** / **architecture.md** / **plan.md** are the contracts; GitHub Flow + CODEOWNERS + branch protection enforce one-owner-per-directory and all-owner sign-off to change a frozen interface. See `CONTRIBUTING.md` and the site's **Collaborate** page.

- A model-agnostic memory harness (4 modules) integrated with OpenCode.
- Three indexed storage backends behind one interface.
- The async dreaming component with governance & pruning.
- A reproducible eval harness (shared runner + cost gates) + baselines on five benchmarks.
- A results scoreboard testing the hypothesis (logged via `results.json` / GitHub Actions).
- The project site (GitHub Pages) documenting all of the above.